MODUL 6 · TYPESCRIPT DALAM NEXT.JS

Alias Tipe



Memberi nama pada tipe agar bisa dipakai ulang, dibaca, dan dirawat dengan mudah

Apa itu Alias Tipe?

Sebuah nama untuk sebuah tipe, ditulis dengan type NamaTipe = ...

Ditulis sekali, dipakai di banyak tempat
Kalau tipenya berubah, cukup ubah di satu tempat

Nama yang bermakna
Status lebih mudah dibaca daripada "aktif" | "nonaktif" | "selesai" yang ditulis berulang

Pesan error ikut memakai nama alias
Error menyebut 'Status', bukan deretan nilai union yang panjang

Tanpa Alias vs Dengan Alias

Union yang sama ditulis tiga kali, lalu diringkas menjadi satu nama.

alias.ts

```
// Tanpa alias: union yang sama ditulis berulang
function tampil(s: "aktif" | "nonaktif" | "selesai") {}
function ubah(s: "aktif" | "nonaktif" | "selesai") {}

// Dengan alias: ditulis sekali, dipakai di mana saja
type Status = "aktif" | "nonaktif" | "selesai";

function tampilStatus(s: Status) {}
function ubahStatus(s: Status) {}
```

Alias Bisa untuk Tipe Apa Saja

Inilah kelebihan type dibanding interface di sub-bab 4.

1. **Union: type ID = number | string**
   Nilai boleh salah satu dari beberapa tipe
2. **Literal union: type Ukuran = "kecil" | "besar"**
   Nilai hanya boleh salah satu dari pilihan yang terdaftar
3. **Objek: type Pengguna = { id: ID; nama: string }**
   Bentuk objek, sama seperti interface
4. **Array: type Daftar = Pengguna[]**
   Kumpulan data dengan bentuk yang sama
5. **Fungsi: type Sapa = (nama: string) => string**
   Bentuk fungsi: parameter dan nilai kembaliannya

Berbagai Jenis Alias

Satu contoh untuk tiap jenis, dengan komentar di atasnya.

jenis-alias.ts

```ts
// union
type ID = number | string;

// objek
type Pengguna = { id: ID; nama: string; email: string; umur: number };

// array
type Daftar = Pengguna[];

// fungsi
type Sapa = (nama: string) => string;
const sapa: Sapa = (nama) => nama.toUpperCase();
```

Derivative Type

Pick, Omit, dan Partial adalah bantuan bawaan TypeScript (utility types) untuk membuat **tipe dari tipe lain.**

Mulai dari satu tipe asal

Misalnya Pengguna dengan id, nama, email, dan password

Turunkan, jangan salin

Kebutuhan lain (tampilan ringkas, form, update) dibuat dari tipe asal itu

Perubahan di tipe asal ikut mengalir

Tambah satu properti di Pengguna, semua turunannya otomatis menyesuaikan

Pick: Ambil Sebagian Properti

Pick<Tipe, "properti"> membuat tipe baru yang hanya berisi properti yang kamu sebutkan.

Pengertian

Dari sebuah tipe, ambil hanya beberapa properti. Yang tidak disebut dibuang

Kapan dipakai

Saat sebuah bagian aplikasi cuma butuh sedikit data, misalnya kartu atau daftar ringkas

Cara menyebut properti

Satu: "id". Beberapa: "id" | "nama" (dipisah tanda |)

Contoh Pick

Ringkas hanya berisi id dan nama. Properti lain ditolak.

pick.ts

```ts
type Pengguna = { id: number; nama: string; email: string; password: string };

type Ringkas = Pick<Pengguna, "id" | "nama">;
// sama dengan: { id: number; nama: string }

const a: Ringkas = { id: 1, nama: "Ani" };
// OK

const b: Ringkas = { id: 1, nama: "Ani", email: "ani@mail.com" };
// Error: Object literal may only specify known properties,
// and 'email' does not exist in type 'Ringkas'.
```

Omit: Buang Sebagian Properti

Omit<Tipe, "properti"> mengambil semuanya kecuali properti yang kamu sebutkan.

Pengertian

Kebalikan Pick: ambil semua properti, lalu buang yang disebut

Kapan dipakai

Menyembunyikan data sensitif (password) atau membuat form tanpa properti yang diisi server (id)

Pilih Pick atau Omit?

Yang diambil sedikit: Pick. Yang dibuang sedikit: Omit

Contoh Omit

Dua turunan dari Pengguna yang sama, untuk dua kebutuhan berbeda.

omit.ts

```ts
type Pengguna = { id: number; nama: string; email: string; password: string };

type PenggunaAman = Omit<Pengguna, "password">;
// sama dengan: { id: number; nama: string; email: string }

type FormDaftar = Omit<Pengguna, "id">;
// sama dengan: { nama: string; email: string; password: string }
// id dibuat oleh server, jadi tidak diisi di form

const p: PenggunaAman = { id: 1, nama: "Ani", email: "a@b.c", password: "x" };
// Error: Object literal may only specify known properties,
// and 'password' does not exist in type 'PenggunaAman'.
```

Jebakan Omit

Omit tidak mengecek nama properti. Pick mengeceknya.

jebakan-omit.ts

```ts
type Pengguna = { id: number; nama: string; email: string; password: string };

type Salah = Omit<Pengguna, "passwrd">;
// tidak ada peringatan! Omit diam saja pada nama yang salah ketik
// hasilnya tetap berisi password

type SalahPick = Pick<Pengguna, "nmaa">;
// Error: Type '"nmaa"' does not satisfy the constraint 'keyof Pengguna'.
```

Partial: Semua Properti Jadi Opsional

Partial<Tipe> mengubah setiap properti menjadi boleh tidak diisi.

Pengertian

Semua properti tetap ada, tetapi masing-masing menjadi opsional (diberi tanda ?)

Kapan dipakai

Update sebagian data: kirim hanya field yang berubah, misalnya edit nama tanpa menyentuh email

Tipe nilai tetap dijaga

Kalau nama dikirim, isinya tetap harus string

Contoh Partial

Fungsi update yang menerima sebagian data Pengguna.

partial.ts

```ts
type Pengguna = { id: number; nama: string; email: string; password: string };

// Partial<Pengguna> sama dengan:
// { id?: number; nama?: string; email?: string; password?: string }

function ubahPengguna(id: number, data: Partial<Pengguna>) {}

ubahPengguna(1, { nama: "Budi" });
// OK: cukup kirim yang berubah

ubahPengguna(1, { nama: 5 });
// Error: Type 'number' is not assignable to type 'string'.
```

Ringkasan: Pick, Omit, dan Partial

Hasilnya jika diterapkan pada Pengguna { id, nama, email, password }.

Utility	Artinya	Hasil	Dipakai untuk
Pick<Pengguna, "id" | "nama">	Ambil hanya yang disebut	id, nama	Tampilan atau data ringkas
Omit<Pengguna, "password">	Ambil semua kecuali yang disebut	id, nama, email	Menyembunyikan data sensitif, form tanpa id
Partial<Pengguna>	Semua properti jadi opsional	Semua properti, boleh kosong	Update sebagian data